

2024

DOCUMENTAZIONE TECNICA APP MINIONS WALLET



CARLO MARNA

SERGIO LEMBO

ANDREA VITOLO

GIORGIA POSTIGLIONE

06/06/2024

[Invito Git Hub](#)

[Repository Git Hub](#)

Sommario

Introduzione	4
Obiettivo	4
Esigenze degli Utenti	4
Monitoraggio delle Spese	4
Organizzazione delle Spese	4
Analisi delle Finanze	4
Facilità d'Uso	4
Personalizzazione.....	4
Interfaccia adattiva.....	4
Utilità aggiuntive	5
Requisiti funzionali	5
Registrazione delle Spese	5
Categorizzazione delle Spese.....	5
Analisi delle Statistiche	5
Personalizzazione dell'Interfaccia	5
Requisiti non funzionali	5
Usabilità	5
Prestazioni	6
Persistenza dei Dati	6
Notifiche	6
Vincoli	6
Definizione degli Utenti Finali e User Journey.....	7
Utenti Finali.....	7
User Journey	7
Design	9
Registrazione e Login	11
Registrazione	11
Login	11
Intestazione Schermata	11
DashBoard e Menu	11
DashBoard	11
Menu	12
Nuova spesa	13
Statistiche & Grafici	14
Uscite	15

Progettazione	16
Modello Incrementale: Evoluzione del Modello a Cascata	16
React Native e Tecnologie Cross-Platform	16
JavaScript e TypeScript	16
JSX.....	16
SQL e Expo-SQLite	16
Database	17
Struttura Database.....	17
Dati pre-inseriti.....	17
Componenti.....	17
Registration	17
Login.....	18
Uscita.....	19
Dashboard	21
Nuova spesa.....	23
Grafici	28
Media	31
Intervallo	33
SpesePerCategoria	36
HomeGraficiStatistiche.....	39
Screen Minions Wallat.....	41
Bug Minions.....	45

Introduzione

L'applicazione “Minions Wallet” è stata progettata per fornire agli utenti uno strumento intuitivo e completo per monitorare e gestire le proprie spese quotidiane. L'idea centrale dietro questa applicazione è fornire agli utenti un'esperienza fluida e personalizzabile per registrare, categorizzare e analizzare le proprie spese, consentendo loro di avere una visione chiara e dettagliata delle loro finanze personali.

Obiettivo

L'obiettivo principale dell'applicazione è aiutare gli utenti a mantenere il controllo delle proprie finanze, consentendo loro di registrare e tracciare le proprie spese in modo efficiente e organizzato. L'applicazione mira anche a fornire agli utenti una panoramica delle proprie abitudini di spesa attraverso statistiche dettagliate e grafici intuitivi.

Esigenze degli Utenti

Monitoraggio delle Spese

Gli utenti desiderano un modo semplice ed efficace per registrare e tenere traccia delle proprie spese quotidiane, consentendo loro di monitorare quanto spendono e qual è la destinazione delle loro uscite.

Organizzazione delle Spese

Gli utenti vogliono poter categorizzare le proprie spese in base a diverse categorie personalizzabili, consentendo loro di organizzare e analizzare le loro uscite in modo più dettagliato e significativo.

Analisi delle Finanze

Gli utenti sono interessati a comprendere le proprie abitudini di spesa attraverso statistiche e grafici chiari e intuitivi, permettendo loro di identificare aree di spesa eccessive o potenziali risparmi.

Facilità d'Uso

Gli utenti cercano un'interfaccia utente intuitiva e user-friendly che renda facile e veloce l'inserimento e la modifica delle spese, consentendo loro di gestire le proprie finanze senza sforzo.

Personalizzazione

Gli utenti desiderano poter personalizzare l'applicazione in base alle proprie esigenze e preferenze, consentendo loro di adattare l'esperienza di utilizzo alle loro specifiche necessità finanziarie.

Interfaccia adattiva

Gli utenti devono poter utilizzare l'applicazione in modo agevole ed efficiente sia in landscape mode che in portrait mode su diversi dispositivi come smartphone e tablet.

Utilità aggiuntive

Gli utenti devono poter registrarsi o effettuare il login per poter accedere al loro conto personale dove sono memorizzate tutte le spese precedentemente inserite e dove possono visualizzare tutte le informazioni relative al proprio conto.

In fase di registrazione, l'utente deve selezionare una valuta che sarà la valuta di default del suo conto e ogni spesa creata anche con valuta diversa sarà convertita nella valuta associata al suo conto.

In fase di creazione di una spesa, l'utente oltre che alla categoria può associare uno o più tag alla spesa; questi tag sono a loro volta creabili.

Requisiti funzionali

Registrazione delle Spese

L'applicazione consente agli utenti di registrare facilmente le proprie spese inserendo importo, data, categoria, descrizione, tag e valuta per ciascuna uscita. Una volta inserita la spesa, l'importo sarà convertito nell'importo relativo alla valuta associata al conto dell'utente in modo tale da permettere ad esempio ad un utente con conto in euro di aggiungere una spesa in yen e visualizzare poi sul proprio conto direttamente la conversione in euro.

Categorizzazione delle Spese

Gli utenti possono categorizzare le proprie spese in base a diverse categorie personalizzabili, consentendo loro di organizzare e analizzare le loro uscite in modo più dettagliato, inoltre possono aggiungere uno o più tag alle spese a loro volta creabili.

Analisi delle Statistiche

L'applicazione fornisce agli utenti una varietà di statistiche dettagliate, tra cui le top-k uscite, grafici dell'andamento della spesa nel tempo e grafici a barre per l'importo medio e totale per ogni categoria. Inoltre, permette di visualizzare la categoria con la spesa minima e massima in un lasso temporale e di visionare come le spese sono distribuite tra le varie categorie. Infine, deve essere possibile visionare la spesa media nel giorno/mese/anno corrente.

Personalizzazione dell'Interfaccia

Gli utenti possono personalizzare l'interfaccia dell'applicazione, inclusi il numero di spese recenti visualizzate sulla schermata principale e le categorie di spese disponibili.

Requisiti non funzionali

Usabilità

L'interfaccia deve essere intuitiva e facile da usare; Inoltre, l'app deve adattarsi a diverse dimensioni di schermo e supportare sia la modalità portrait che landscape.

Prestazioni

L'app deve essere reattiva e le operazioni di inserimento, modifica ed eliminazione delle spese devono essere rapide.

Persistenza dei Dati

I dati devono essere salvati in modo persistente, anche alla chiusura dell'app.

Notifiche

L'app deve poter inviare notifiche push per ricordare all'utente di registrare le sue spese.

Vincoli

L'app deve essere sviluppata utilizzando React Native o Flutter.

Definizione degli Utenti Finali e User Journey

Nella fase di definizione degli utenti finali dell'applicazione si utilizzano metodologie come lo User Journey per delineare le caratteristiche principali che dovranno essere implementate per soddisfare le esigenze degli utenti. Questo processo coinvolge l'identificazione dei diversi tipi di utenti che interagiranno con l'applicazione e la comprensione delle loro esigenze, obiettivi e comportamenti.

Utenti Finali

1. **Utente Occasionale:** Questo utente utilizza l'applicazione sporadicamente per registrare spese occasionali o per controllare rapidamente il saldo corrente. Cerca un'esperienza semplice e intuitiva senza necessità di funzionalità avanzate.
2. **Utente Abituale:** Questo utente utilizza l'applicazione regolarmente per registrare tutte le sue spese e monitorare attentamente il proprio budget. Cerca un'applicazione completa con funzionalità avanzate per analizzare le proprie abitudini di spesa.

User Journey

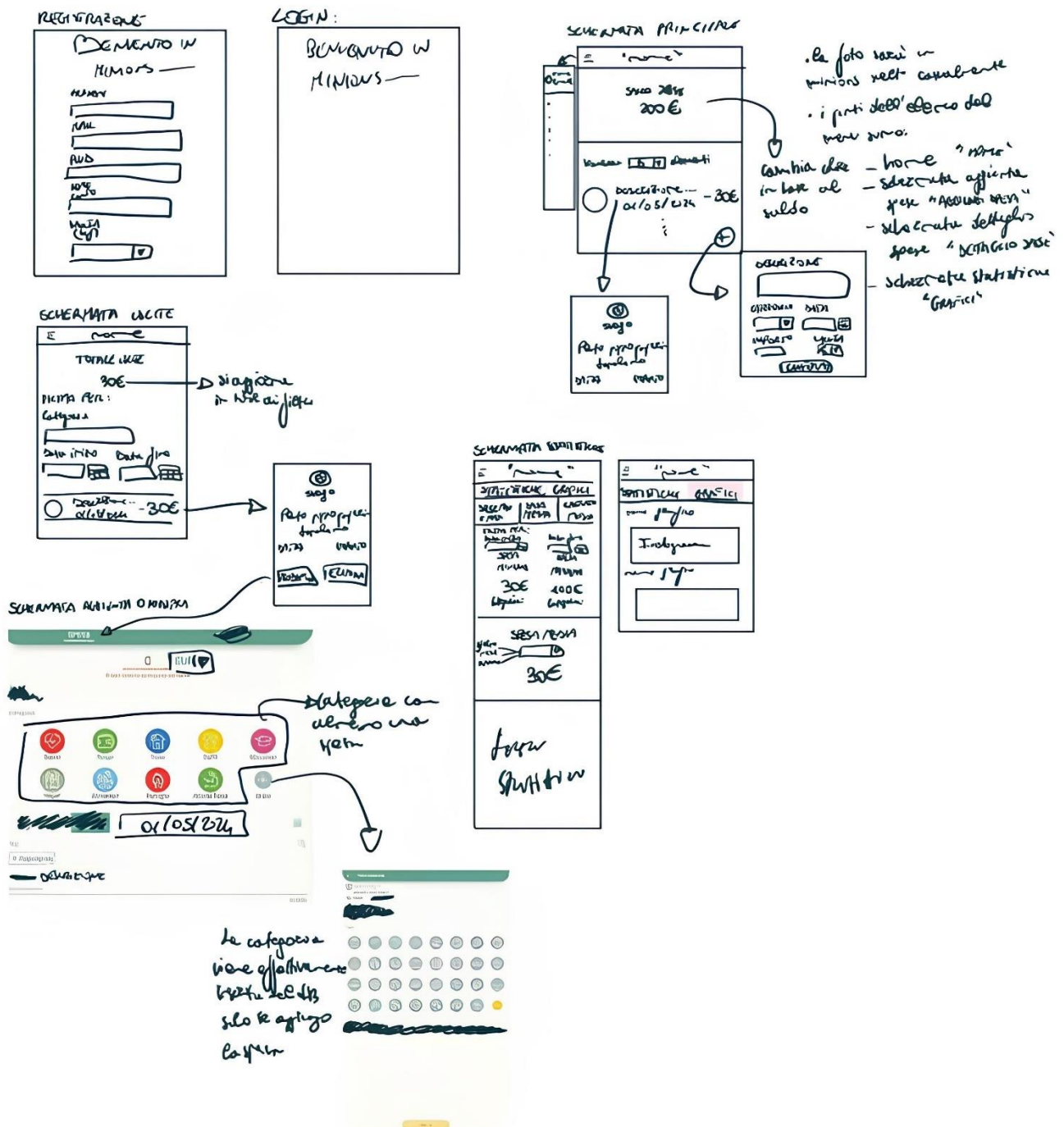
- **Accesso al conto**
 - A. L'utente accede all'applicazione e deve scegliere se registrarsi, inserendo username, mail, password e valuta, o effettuare il login ad un conto esistente cliccando sulla sezione apposita per venir reindirizzato alla sezione di login, dove dovrà inserire username e password riferiti al conto creato in precedenza.
 - B. Dopo aver effettuato la registrazione o il login, l'utente visualizza la homepage e attraverso il menu laterale può navigare tra le schermate.
- **Registrazione della Spesa:**
 - A. L'utente naviga dal menu e seleziona "Nuova spesa" venendo reindirizzato alla sezione apposita;
 - B. Inserisce l'importo, la data, la categoria (o ne crea una nuova), uno o più tag associati a quella spesa o ne crea uno nuovo, una breve descrizione e la valuta della spesa;
 - C. Conferma l'inserimento e la spesa viene aggiunta al registro delle uscite;
- **Visualizzazione del Saldo Corrente:**
 - A. L'utente accede alla schermata principale dell'applicazione e visualizza immediatamente il saldo corrente e tutte le spese da lui sostenute.
- **Aggiunta Rapida di una Spesa:**
 - A. L'utente accede alla schermata principale dell'applicazione;
 - B. Clicca il pulsante per aggiungere una spesa rapida;
 - C. Inserisce l'importo, la data, una categoria già esistente, una breve descrizione della spesa e conferma l'operazione;
- **Esplorazione dei Grafici:**
 - A. L'utente naviga verso la schermata dei grafici per analizzare le proprie abitudini di spesa.
 - B. Esplora i grafici dell'andamento della spesa nel tempo e l'importo medio e totale per ogni categoria.
 - C. Identifica eventuali aree di spesa eccessive o potenziali risparmi.
- **Esplorazione delle Statistiche:**
 - A. L'utente naviga verso la schermata delle statistiche per analizzare le proprie abitudini di spesa.
 - B. Inserisce un periodo di riferimento da analizzare e gli viene mostrato quale categoria ha avuto la spesa massima e minima in quel periodo;

- C. Inoltre, l'utente può scegliere di visualizzare la media delle spese per il giorno/mese/anno corrente;
- D. Infine, può visualizzare come sono distribuite le spese per le varie categorie.
- Gestione delle Spese:
 - A. L'utente naviga verso la schermata delle uscite per visualizzare e gestire tutte le spese registrate.
 - B. Può eliminare o modificare una spesa esistente, oltre a filtrarle per data e categoria.
 - C. In caso di modifica l'utente viene reindirizzato alla pagina di aggiunta spesa, dove visualizzerà tutti i campi già popolati dai dati della spesa selezionata e può decidere cosa modificare e poi cliccare su modifica spesa.
 - D. Dopo aver cliccato su modifica spesa l'utente viene reindirizzato di nuovo alla pagina delle uscite e potrà visualizzare la spesa modificata.
- Personalizzazione dell'Interfaccia:
 - A. L'utente personalizza l'interfaccia dell'applicazione secondo le proprie preferenze.
 - B. Modifica il numero di spese recenti visualizzate sulla schermata principale e personalizza le categorie di spese disponibili.

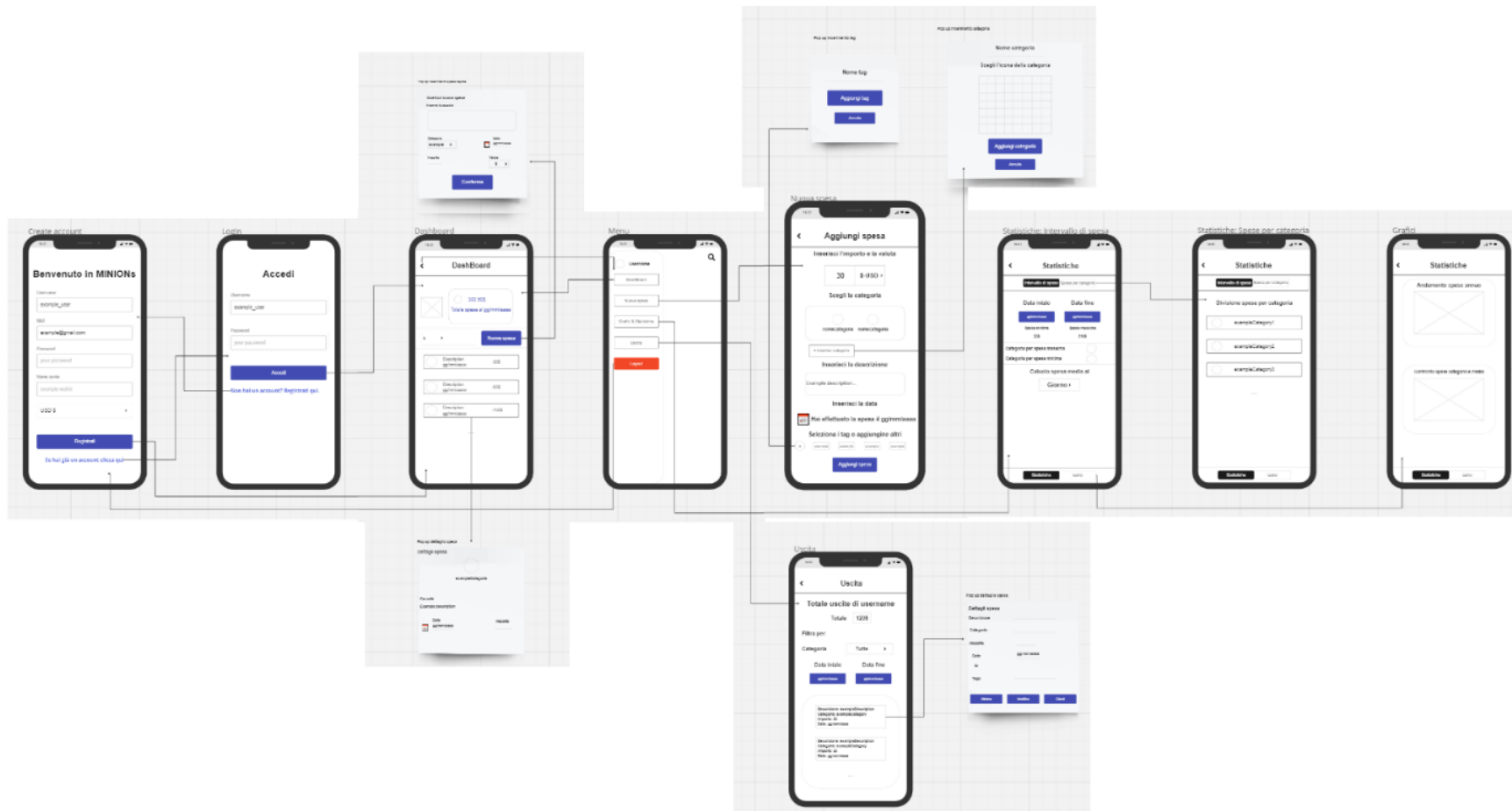
Design

La fase successiva è la fase di design: in questa fase si stabiliscono le fondamenta sia per l'esperienza utente (UX) che per l'interfaccia utente (UI). Per descrivere l'interfaccia utente vengono usati i wireframes, delle rappresentazioni schematiche utili nella fase di progettazione di un'applicazione per definire la struttura e il layout delle diverse schermate.

Una prima bozza dell'applicazione, senza l'utilizzo di tools per creare schemi quanto più fedeli alla realtà, è la seguente:

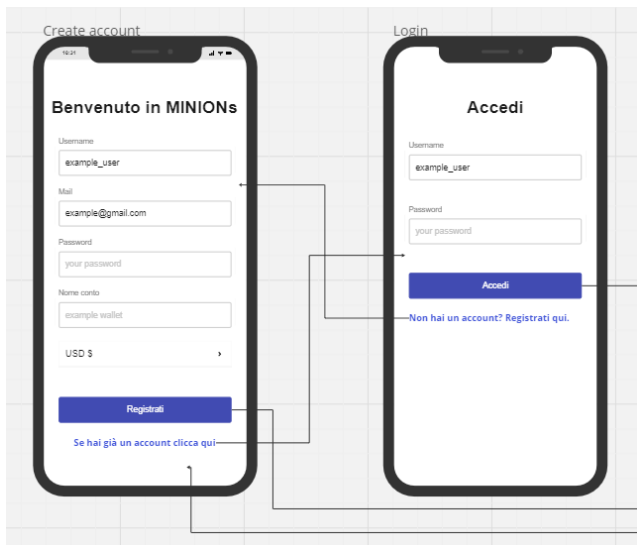


Il wireframe ad alta fedeltà dell'applicazione è il seguente:



Registrazione e Login

Registrazione



La pagina di registrazione è la prima ad essere visualizzata dall'utente quando viene aperta l'applicazione. Permette a quest'ultimo di registrarsi inserendo le informazioni di base, quali username, mail, password, nome del conto e valuta di default scelta. I primi quattro campi sono testuali mentre l'ultimo è un menu a tendina per l'inserimento della valuta associata al conto. Cliccando sul bottone “Registrati” si verrà reindirizzati alla DashBoard.

Se l'utente ha già un account, tramite l'apposito link potrà essere reindirizzato alla pagina di Login.

Login

La pagina di Login può essere raggiunta dall'utente tramite il link posto sotto al bottone “Registrati” nella pagina di registrazione.

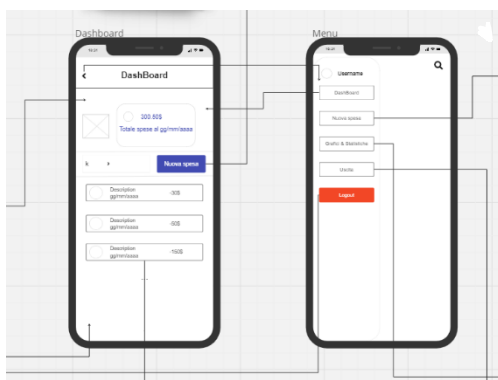
È composta da due campi testuali in cui l'utente deve inserire rispettivamente lo username e la password. È presente infine il bottone “Accedi”, che, quando viene cliccato, reindirizza l'utente alla DashBoard. Anche in questa pagina è presente un link posizionato sotto il bottone che permette all'utente di ritornare alla pagina dedicata alla registrazione.

Intestazione Schermata

Tutte le schermate caricate dopo la fase di registrazione o login avranno una intestazione comune in cui sarà il nome della pagina e l'icona del menu, e che se premuta, fa comparire quest'ultimo; La descrizione del menu sarà fatta in seguito.

DashBoard e Menu

DashBoard



Completata la fase di registrazione o la procedura di Login, l'utente visualizzerà la pagina che possiamo considerare come home page della nostra applicazione, cioè la DashBoard.

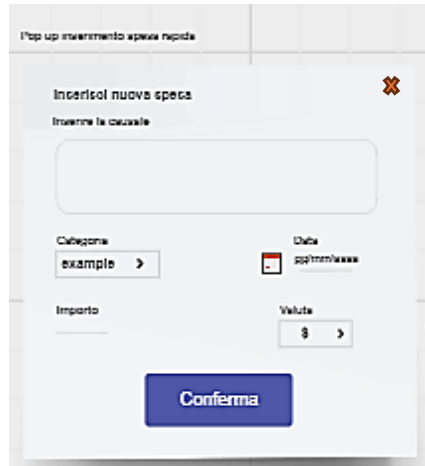
Possiamo considerare questa pagina come se fosse suddivisa in due sezioni distinte:

- **Saldo**

È l'area in cui l'utente può visualizzare il totale delle spese, calcolato sommando tutti gli importi delle spese sempre nella valuta di default scelta, alla data corrente.

- **Spese**

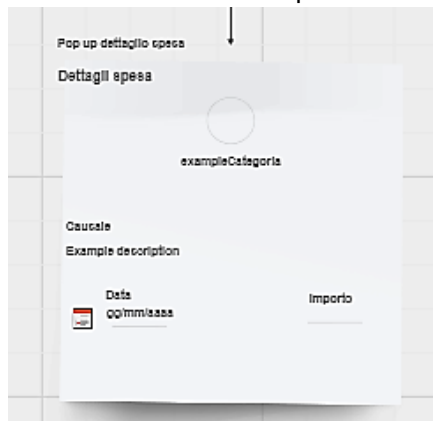
È la sezione in cui l'utente può visualizzare le ultime k spese, scegliendo il valore di k dal menu a tendina posto in alto a sinistra (i valori di k possibili sono 10, 25, 50 e 100), ed in



più può effettuare un inserimento rapido di una spesa cliccando sull'apposito bottone che farà comparire il pop up dedicato.

Tramite quest'ultimo, l'utente potrà inserire una nuova spesa, ma non con tutte le funzionalità implementate invece nella pagina dedicata all'aggiunta delle spese. Il pop up è composto da: un campo testuale in cui l'utente inserisce la descrizione (causale) della spesa; un menu a tendina in cui saranno presenti tutte le categorie già create in precedenza dall'utente e, tra queste, potrà scegliere quella da associare alla nuova spesa, ma non potrà inserirne una nuova, infatti, tale

funzionalità sarà implementata invece nella pagina "Nuova Spesa"; un campo di inserimento testuale per inserire l'importo della spesa; la valuta di default, che



nell'inserimento rapido della spesa non può essere modificata, e il campo data, che se cliccato farà comparire il calendario da cui poter selezionare la data relativa alla spesa.

Cliccando sul bottone "Conferma", la spesa inserita dall'utente verrà aggiunta e potrà essere visualizzata nella DashBoard.

Un'altra funzionalità non presente nel pop up di inserimento rapido della spesa è l'aggiunta dei tag, che sarà possibile solo nella pagina "Nuova spesa".

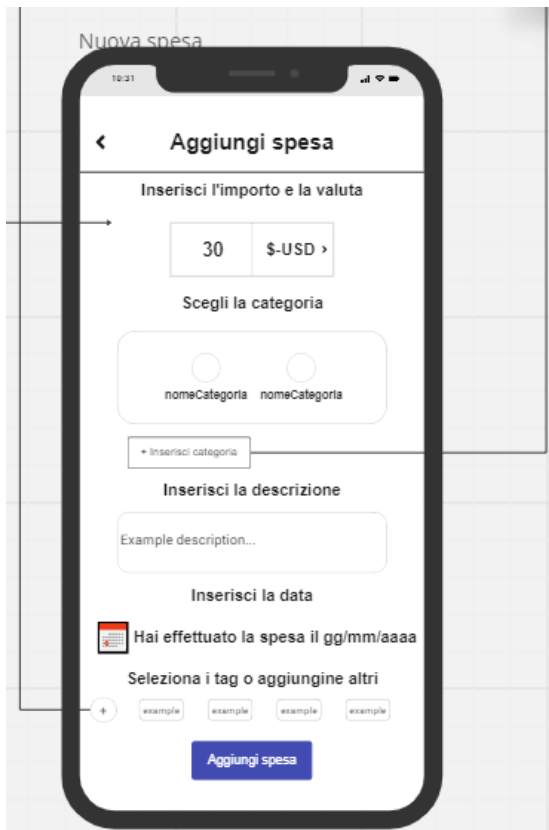
Cliccando, invece, su una delle spese visualizzate

comparirà un pop di descrizione della spesa, utile quando la causale inserita è troppo lunga per essere visualizzata interamente nel box dedicato alla spesa.

Menu

Il menu è il componente dell'app che ci permette di navigare tra le varie pagine. È composto da: un'intestazione, in cui è presente lo username dell'utente e l'immagine randomica associatagli; quattro bottoni che, se cliccati, portano alle rispettive quattro pagine (DashBoard, Nuova spesa, Statistiche & Grafici e Uscite) e dal bottone di Logout che riporterà l'utente alla pagina di registrazione, disconnettendolo.

Nuova spesa



La pagina di Nuova Spesa è quella dedicata all'aggiunta di una nuova spesa con tutte le funzionalità fornite all'utente. Possiamo considerarla come composta da cinque sezioni:

- **Importo**
In quest'area è presente un campo di inserimento testuale per inserire l'importo della spesa e un menu a tendina in cui l'utente può scegliere la valuta associata alla spesa tra quelle a disposizione.
- **Categoria**
In quest'area vengono visualizzate tutte le categorie precedentemente create dall'utente (quelle che hanno almeno una spesa associata), tra cui può scegliere quella da associare alla spesa che sta inserendo. In più, è presente il bottone "Inserisci categoria", che se cliccato fa comparire il pop up che permette all'utente di inserire una nuova categoria.

Nel pop up è presente il campo per inserire il nome della categoria ed una griglia con tutte le icone, tra cui l'utente

può scegliere quella da associare alla nuova categoria.

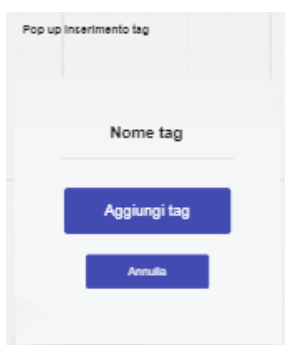
- **Descrizione**

In quest'area è presente il campo di inserimento testuale in cui l'utente inserisce la descrizione associata alla spesa.

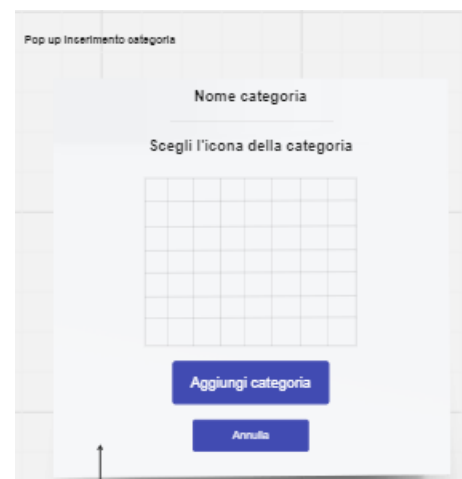
- **Data**

In quest'area è presente l'icona del calendario, che, se cliccata farà comparire quest'ultimo da cui l'utente sceglierà la data da associare alla nuova spesa; questa può essere visualizzata nel campo testuale presente anch'esso in questa sezione.

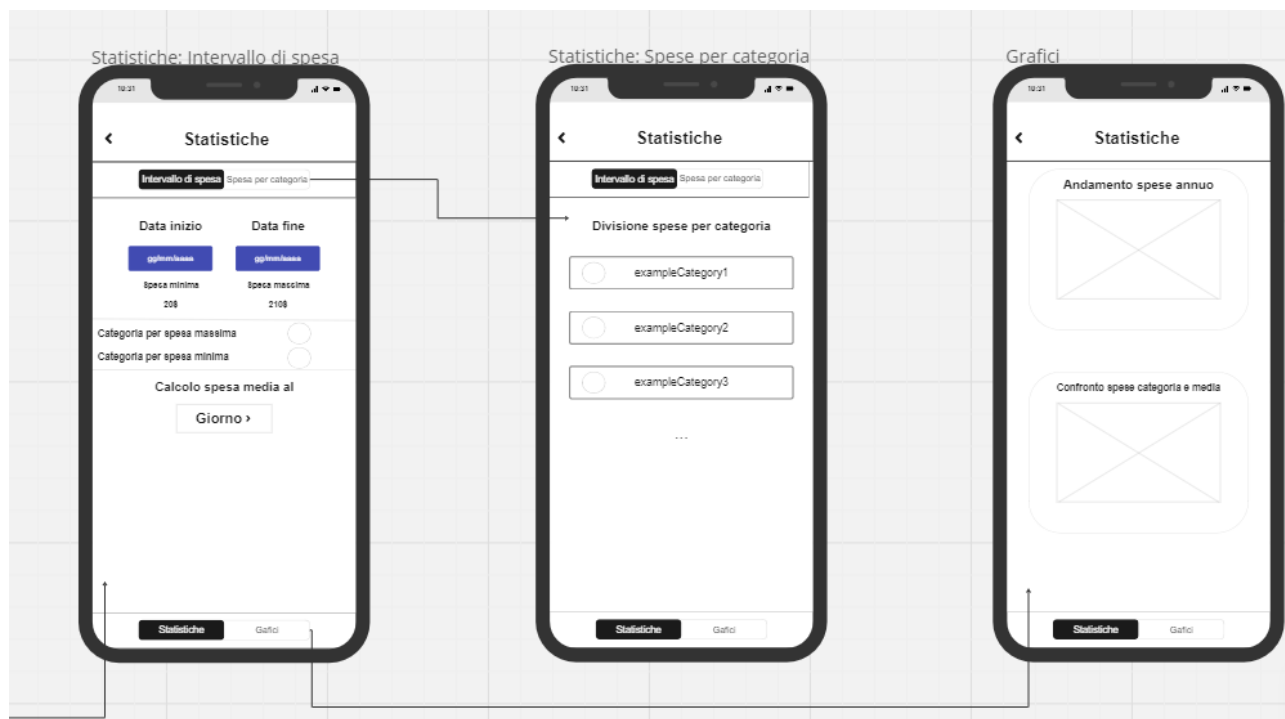
- **Tag**



In quest'area l'utente può visualizzare i tag precedentemente inseriti che possono essere associati alla nuova spesa; in più, è presente un bottone che se cliccato farà comparire il pop up che permette all'utente di inserirne uno nuovo, specificandone il nome. Al termine della pagina è presente il bottone "Aggiungi spesa", che, se cliccato farà uscire un Alert, il cui contenuto cambia in base a se la spesa viene aggiunta correttamente o no, nel caso in cui manchino dei campi obbligatori.



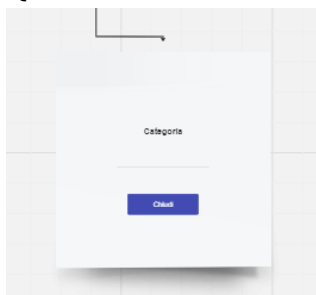
Statistiche & Grafici



La pagina di Statistiche & Grafici è in realtà composta da tre sezioni differenti, tutte raggiungibili tramite le due tap bar; infatti, la tap bar posta subito dopo l'intestazione (che, come nelle altre pagine, è composta da nome della pagina ed icona del menu) è quella che permette all'utente di passare dalla sezione di "HomeMediaMinMax" a quella di "Spese per categoria", mentre quella posta in bassa serve a passare dalla sezione delle Statistiche sopra citate a quella dei Grafici.

- **Statistiche: Intervallo di spesa**

Quest'area è suddivisa in due sottosezioni:



- La prima è quella che permette all'utente di visualizzare la categoria della spesa massima e quella della spesa minima in un arco temporale, selezionato tramite i calendari associati rispettivamente al bottone di "Data inizio" e a quello di "Data fine". Inoltre vi sono due Bottoni uno per l'icona della categoria con la spesa massima e uno per l'icona per la categoria con la spesa minima, se cliccati, comparirà un "pop Up" in cui vi è il nome della categoria associata all'icona.

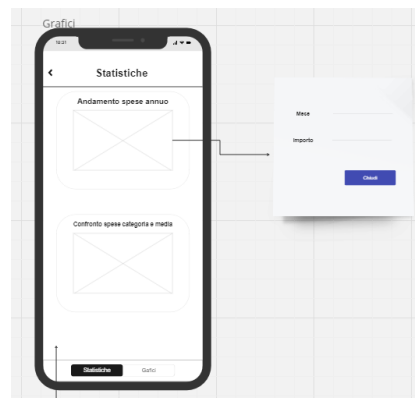
- La seconda è quella in cui l'utente può visualizzare la spesa media del giorno, del mese o dell'anno corrente; l'opzione desiderata può essere selezionata tramite l'apposito menu a tendina.

- **Statistiche: Spesa per categoria**

In quest'area l'utente può visualizzare tutte le categorie con la rispettiva percentuale di spese associate.

- **Grafici**

In quest'area sono presenti due grafici: il primo è un line chart, che mostra l'andamento delle spese in un anno solare, mentre il secondo è un grafico a barre che mostra per ogni categoria il totale e la media delle spese. Per quanto riguarda il line chart se viene cliccato un elemento del grafico compare un pop Up contenente le informazioni associate all'elemento cliccato.



Uscite



È la pagina in cui l'utente può visualizzare tutte le spese inserite. È composta da due sottosezioni:

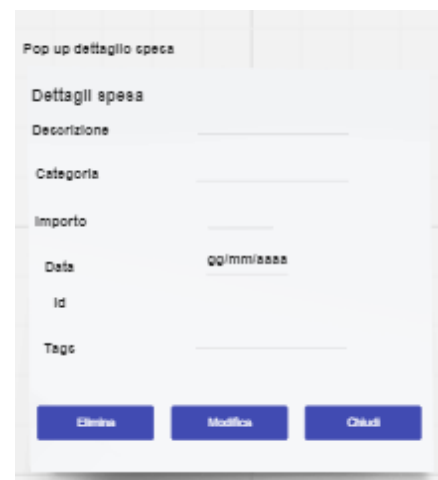
- **Filtri**

È la sezione in cui l'utente può inserire i filtri per la visualizzazione delle categorie; è composta da un menu a tendina per scegliere la categoria di interesse e due bottoni per selezionare la data di inizio e la data di fine dell'arco temporale di cui si vogliono visualizzare le spese; per entrambi comparirà lo spinner per far selezionare la data all'utente.

- **Spese**

È la sezione in cui viene visualizzato l'elenco delle spese sulla base dei filtri selezionati. Cliccando sulla box della spesa, comparirà un pop up con la descrizione dettagliata della spesa e tre bottoni, che sono rispettivamente per eliminare o modificare la spesa oppure per chiudere il pop up. Cliccando sul

bottone di "Modifica" l'utente verrà reindirizzato alla pagina di Nuova spesa, in cui però i campi sono già compilati con i dati della spesa da modificare.



Progettazione

Durante la fase di progettazione dell'applicazione di “Minions Wallet”, si adotta un approccio basato sul modello incrementale, che rappresenta un'evoluzione del tradizionale modello a cascata. Questo approccio consente di suddividere il progetto in parti più piccole, che vengono sviluppate separatamente in parallelo e poi integrate gradualmente. Inoltre, si fa uso di tecnologie cross-platform, come React Native e Expo, per garantire la compatibilità dell'applicazione su diverse piattaforme mobili. L'app verrà sviluppata per la piattaforma Android.

Modello Incrementale: Evoluzione del Modello a Cascata

Il modello incrementale adottato è un'evoluzione del modello a cascata, che prevede la suddivisione del progetto in fasi più piccole e gestibili, sviluppate e testate separatamente prima di essere integrate nel sistema completo. Questo approccio consente di contenere il rischio associato allo sviluppo software, in quanto eventuali problemi o errori in una fase non compromettono l'intero progetto. Inoltre, permette una maggiore flessibilità e adattabilità alle esigenze degli utenti e alle evoluzioni del mercato, consentendo un miglioramento continuo e adattivo dell'applicazione nel tempo.

React Native e Tecnologie Cross-Platform

Per sviluppare l'applicazione mobile, si fa ampio uso di React Native, un framework che consente di scrivere codice in JavaScript che viene poi tradotto in codice nativo per ciascuna piattaforma come iOS e Android. Inoltre, si utilizza Expo, una piattaforma open-source che semplifica lo sviluppo di applicazioni React Native fornendo strumenti, librerie e servizi aggiuntivi. Expo semplifica la gestione delle risorse, la distribuzione dell'applicazione e offre un set di componenti predefiniti e funzionalità pronte all'uso. Utilizzando React Native, è possibile sviluppare un'applicazione intuitiva, con performance simili a quelle di un'app nativa, e altamente personalizzabile, che si adatta alle esigenze degli utenti e alle caratteristiche specifiche di ciascuna piattaforma.

Nei paragrafi successivi verranno descritti i linguaggi utilizzati per la realizzazione dell'App.

JavaScript e TypeScript

JavaScript è il linguaggio di programmazione principale utilizzato nello sviluppo dell'applicazione, grazie alla sua ampia adozione e alla sua flessibilità. Inoltre, TypeScript, una versione tipizzata di JavaScript, viene utilizzato per migliorare la qualità del codice e garantire una migliore manutenibilità e scalabilità dell'applicazione.

JSX

JSX, o JavaScript XML, è una sintassi di estensione per JavaScript utilizzata principalmente con React, una libreria JavaScript per costruire interfacce utente. JSX permette di scrivere il codice che assomiglia a HTML all'interno dei file JavaScript.

SQL e Expo-SQLite

Per la gestione dei dati persistenti, viene utilizzato SQLite, un database relazionale leggero e performante, che consente di memorizzare e recuperare i dati dell'applicazione in modo efficiente. In particolare, Expo-SQLite è una libreria che semplifica l'accesso a SQLite nell'ambiente di sviluppo di Expo, fornendo un'interfaccia semplice e intuitiva per eseguire query SQL e gestire il database all'interno dell'applicazione. Un'altra libreria che permette di creare un database e dunque assicurare

la persistenza dei dati è SQLite di react-native-sqlite-storage; non è stato possibile usare questa libreria perché non è supportata da Expo.

Database

Struttura Database

Il database è composto da **sette tabelle principali**:

- Valuta: Memorizza le informazioni sulle valute supportate, tra cui sigla, nome e simbolo.
- Icona: Memorizza i percorsi delle icone utilizzate per le categorie e i conti;
- Utente: Memorizza le informazioni sugli utenti, tra cui username, email e password;
- Conto: Memorizza le informazioni sui conti, tra cui nome, sigla della valuta e username del proprietario;
- Categoria: Memorizza le informazioni sulle categorie di spesa, tra cui nome, percorso dell'icona e ID del conto a cui è associata;
- Spesa: Memorizza le informazioni sulle spese effettuate dall'utente, tra cui l'id, l'importo, data della spesa e la categoria e il conto a cui è associata la spesa.
- Tag: Memorizza i tag associati alle spese.

Dati pre-inseriti

Oltre che a dati necessari al funzionamento dell'app come l'inserimento delle valute e dei path delle immagini da assegnare alle categorie, in fase di creazione è stato predisposto un account di test con delle spese e categorie pre-inserite, per accedervi è necessario fare il login con username '**prova_app**' e password '**12345678**'.

Componenti

I componenti sono i blocchi fondamentali di qualsiasi applicazione React. Sono simili alle funzioni JavaScript, ma lavorano in modo indipendente; accettano un oggetto props come argomento e restituiscono un elemento React che compone l'interfaccia utente.

Per ogni componente abbiamo utilizzato la funzione `dimension.get` per ottenere in maniera dinamica la larghezza e l'altezza della finestra su cui viene visualizzata l'app in questo modo gli stili si applicano in maniera dinamica e l'app risulta utilizzabile sia in verticale che in orizzontale e su schermi di dimensioni maggiori come i tablet o minori come gli smartphone più piccoli.

Registration

Importazioni

Tra le varie importazioni di base del componente c'è il selettore di valori (Componente **Picker** da `@react-native-picker/picker`) che servirà a selezionare la valuta. Il componente riceve come props `navigation`, `database` e `onLogin`; quest'ultima serve a richiamare una funzione in app che setterà l'username dell'utente e l'id del conto per gli altri componenti dopo aver effettuato la registrazione e quindi, dopo la creazione dell'utente e del conto.

Stati

Tra gli stati definiti abbiamo `username`, `email`, `password` e `accountname`, di cui i significati sono abbastanza ovvi; inoltre, c'è `currencies` che viene utilizzato per caricare le valute presenti nel database dopo il caricamento di quest'ultimo e viene utilizzato dal picker per avere una lista di valute tra cui l'utente può scegliere. `SelectedCurrency` è, invece, utilizzato dal picker per selezionare la valuta e creare

il conto su quella valuta e ha come valore di default l'euro. `IsDatabaseInitialized` è una variabile di comodità per un problema riscontrato.

Funzioni principali

- **fetchCurrencies():** serve a caricare dal db tutte le valute e a settarle nella variabile `currencies`; viene richiamata ogni qual volta c'è un cambiamento nel db attraverso `useEffect()`.
- **handleRegistration():** è la funzione che si occupa di gestire il meccanismo della registrazione: in primis verifica che tutti i campi siano stati compilati; poi verifica che l'email abbia una sintassi corretta e al suo interno invoca anche un'altra funzione, `registrazioneUtente()`, che restituisce un messaggio. Il contenuto di questo messaggio serve a verificare che nel tentativo di inserire i dati nel database non sia stato trovato uno username o una mail uguali già presenti. Se tutto è andato a buon fine, mostra un alert e si viene reindirizzati alla homepage.
- **registrazioneUtente():** è la funzione che si occupa di effettuare gli inserimenti nel database dei dati inseriti dall'utente, restituendo un messaggio specifico in caso di username duplicato o email già utilizzata; inoltre, all'interno di essa vengono settati i parametri di `onLogin` dopo che l'inserimento è andato a buon fine, per poter passare al componente principale *App* lo username e l'`id_conto` dell'utente.

Interazioni con l'utente e utilizzo

L'utente vede questa pagina come prima pagina dell'app, ha la possibilità o di effettuare la registrazione inserendo username, email, password e scegliendo la valuta di default del suo conto, oppure, nel caso abbia già un account, può cliccare sul testo apposito per essere reindirizzato alla pagina di Login.

Problemi incontrati e soluzioni adottate

Partiamo dicendo che ci sarebbe piaciuto implementare la cifratura delle password, ma abbiamo avuto problemi con la libreria e per questioni di tempo abbiamo fatto un passo indietro. In questa pagina il problema più grande è stato che, essendo la prima ad essere visualizzata mentre in background il database viene inizializzato, la query che carica le valute andava sempre in errore poiché veniva effettuata prima che le valute venissero effettivamente inserite. Per ovviare a questo problema, abbiamo utilizzato una variabile di servizio `isDatabaseInitialized` che viene settata a true quando il database viene effettivamente inizializzato sfruttando il meccanismo dello `useEffect`. La funzione `fetchCurrencies()` verifica che il database sia inizializzato e solo allora effettua la query.

Login

Importazioni

Oltre ai componenti base, tra le props ci sono `navigation`, `database` e `onLogin` che, come in *Registration*, serve a fornire al componente *App* i dati relativi ad username e idconto dell'utente che effettua il login.

Stati

In questo componente gli unici stati sono username e password.

Funzioni principali

- **handleLogin():** è l'unica funzione del componente e gestisce tutta la logica di log partendo dal controllo dei campi non vuoti, effettua il controllo delle credenziali inserite con i dati presenti nel database, restituisce ad *App* idconto e username e reindirizza alla Dashboard in caso di esito positivo.

Uscita

Importazioni

Oltre che ai componenti base di React, c'è sia il Picker che il DateTimePicker che serviranno a selezionare Categoria e Data per filtrare le uscite. Tra le props ci sono sempre navigation, database, idconto e username che permettono di vedere le uscite associate all'utente in sessione.

Stati

Tra gli stati ci sono: *total*, che mantiene il totale di tutte le uscite che rispettano i filtri; *category*, che serve ad avere in una variabile la categoria attualmente selezionata dal picker delle categorie; *start date* ed *end date*, che fanno la stessa cosa di *category*, ma per le date; *categories*, per memorizzare in un array locale le categorie presenti nel database associate all'utente specifico; *spese* per avere un array locale di tutte le spese presenti nel database dell'utente specifico che rispettano i filtri. In più ci sono una serie di stati di servizio per la visualizzazione dei vari picker e del modal.

Funzioni principali

- **fetchCategories()**: ogni volta che il database cambia viene richiamata e server a selezionare le categorie dal database.
- **calculateTotal()**: ogni volta che o il database o la categoria selezionata o le date selezionate cambiano ricalcola il totale selezionando dal database tutte le spese associate all'utente che matchano con la categoria selezionata e che sono contenute nell'intervallo delle due date selezionate.
- **fetchSpese()**: ogni volta che o il database o la categoria selezionata o le date selezionate cambiano, seleziona tutte le spese associate all'utente che matchano con la categoria selezionata e che sono contenute nell'intervallo delle due date selezionate e le inserisce nell'array locale di spese che verrà visualizzato dalla Flatlist.
- **deleteSpesa(spesaDaEliminare)**: riceve come parametro la spesa da eliminare in quanto per richiamare questa funzione è necessario aver selezionato la spesa dalla Flatlist; una volta effettuata l'eliminazione della spesa dal database, la funzione richiama *fetchSpese()* per aggiornare la lista, in quanto non sempre il cambiamento nel database veniva riconosciuto, ed inoltre verifica che ci siano spese associate alla categoria della spesa eliminata, in quanto da requisito se una categoria non ha spese associate, allora deve essere eliminata. Se non trova spese per quella categoria, ciò vuol dire che la spesa eliminata era l'ultima di quella categoria, allora elimina la categoria dalla tabella del database delle categorie e richiama *fetchCategories()* per aggiornare le categorie selezionabili dal Picker.
- **takeTag()**: questa funzione serve a selezionare i tag associati alla spesa selezionata, se presenti, per memorizzarli in una variabile di stato e renderli visibili nel modal associato alla spesa selezionata.

Problemi incontrati e soluzioni adottate

I problemi principali sono stati di sincronizzazione col database in quanto dopo un'eliminazione non veniva eliminata la spesa dalla lista delle spese; per risolvere ciò ogni qual volta c'è un'eliminazione viene richiamata la funzione per aggiornare le spese e quella per aggiornare le categorie.

Un altro problema è sorto per selezionare i tag; poiché la funzione dei tag deve per sua natura essere richiamata dopo aver cliccato sulla spesa nella Flatlist, il risultato della query non valorizzava in tempo la variabile di stato *tags* in quanto il Modal veniva visualizzato prima e il campo tag risultava quindi null. Per risolvere ciò, la funzione *takeTag()* viene richiamata per mezzo di uno *useeffect()* ogni volta che la spesa selezionata cambia; così facendo è stato risolto il problema della sincronizzazione.

Interazioni con l'utente e utilizzo

L'utente accede a questa pagina per visualizzare un elenco completo e filtrabile di tutte le sue uscite. Ha la possibilità di filtrare le uscite per categoria e per data selezionando una data di inizio e una di fine, ed una volta fatto ciò automaticamente vedrà a schermo tutte le uscite che rispettano i filtri e il totale di esse, nella valuta che ha selezionato come valuta di default del suo conto. Scorrendo la lista delle uscite è possibile cliccare su una di esse per avere informazioni in più come i tags; in più, l'utente può eliminare quella spesa definitivamente o modificarla. Cliccando su modifica, viene portato alla pagina di inserimento spese e può modificare qualsiasi parametro a sua scelta e, una volta fatto ciò, cliccando su "Applica modifica" viene riportato sulla pagina *Uscite* e visualizzerà le info aggiornate.

Dashboard

Il componente **Dashboard** è il componente principale visualizzato immediatamente dopo il login o la registrazione dell'utente. Fornisce una panoramica iniziale delle spese inserite e permette una gestione rapida delle nuove spese. Ecco una descrizione dettagliata del componente:

Proprietà del Componente

- **database:** Un oggetto che rappresenta il database SQLite, utilizzato per recuperare e salvare le spese.
- **idConto:** Una stringa che rappresenta l'ID del conto per il quale devono essere visualizzate tutte le spese precedentemente inserite.

Funzionalità Principali

- **Panoramica delle Spese**
 - **Elenco delle Spese:** Viene visualizzato un elenco delle spese inserite precedentemente. Ogni spesa mostra informazioni essenziali come la data, l'importo e una breve descrizione.
 - **Popup Dettagli:** Cliccando su una spesa, si apre un popup che mostra un dettaglio completo delle informazioni di quella specifica spesa, inclusa la categoria e la descrizione completa della spesa.
 - **Numero di Spese Visualizzate:** Un picker permette di selezionare il numero di spese da visualizzare nell'elenco. I valori preimpostati sono, 10, 25, 50 e 100.
- **Inserimento Rapido delle Spese**
 - **Form di Inserimento Rapido:** Nella dashboard è presente un modulo per l'inserimento rapido delle spese. Questo modulo permette di inserire nuove spese con i seguenti campi:
 - **Importo:** Campo per inserire l'importo della spesa, con la valuta di default specificata durante la registrazione.
 - **Categoria:** Picker per selezionare la categoria della spesa da una lista di categorie che creerà l'utente stesso. Non è possibile creare nuove categorie direttamente da questo modulo; per farlo, è necessario accedere alla sezione dedicata Nuova Spesa.
 - **Data:** Campo per selezionare la data della spesa.
 - **Descrizione:** Campo per aggiungere una breve descrizione della spesa.

Stati del componente

La Dashboard utilizza diverse variabili di stato per gestire le informazioni relative alle spese, alla valuta di default alle categorie ecc...:

1. **LimitElementiFlatList:** Memorizza il numero delle spese da visualizzare;
2. **SaldoConto:** Memorizza il saldo spese del conto corrente;
3. **Valuta:** Memorizza la valuta di default scelta dall'utente in fase di registrazione;
4. **ModalVisible:** Memorizza lo stato del modal del riepilogo spese;
5. **SpesaModalVisible:** Memorizza lo stato del modal dell'inserimento rapido di una nuova spesa;
6. **SelectedSpesa:** Memorizza tutte le informazioni inerenti alla spesa di cui l'utente vuole visualizzare il riepilogo;
7. **Spese:** Memorizza tutte le spese prelevate dal database associate al conto corrente dell'utente loggato;

8. `IsInsertNewSpesa`: Memorizza uno stato che cambia ogni qualvolta che viene inserita una nuova spesa, così da scatenare eventi;
9. `Data`: Memorizza la data selezionata dall'utente;
10. `ViewDatePicker`: Memorizza uno stato che scatena l'apertura e la chiusura del `DateTimePicker`;
11. `Causale`: Memorizza la descrizione/causale associata a una spesa in fase di inserimento;
12. `CategoriaSelezionata`: Memorizza la categoria che l'utente seleziona in fase di inserimento;
13. `Categoria`: Memorizza una lista di categorie inserite dall'utente nella sezione apposita;
14. `Importo`: Memorizza l'importo della spesa che si sta inserendo;

Nuova spesa

La pagina “Nuova spesa” è quella dedicata all’aggiunta delle spese dell’utente e alla modifica di una spesa già esistente. I campi che l’utente deve obbligatoriamente inserire prima che l’operazione di aggiunta spesa vada a buon fine sono tutti tranne i tag, che sono invece opzionali. Sia le categorie che i tag saranno associati al conto dell’utente e, quindi, utenti diversi non vedranno mai né le stesse categorie né gli stessi tag. Per far ciò, utilizziamo i dati prelevati dal database dell’utente.

Proprietà di Nuova spesa

- **database**: l’oggetto che rappresenta il database e che serve al componente per recuperare i dati relativi all’utente
- **idConto**: la stringa contenente l’identificativo del conto dell’utente che ci permette di filtrare le informazioni prelevate dal database
- **route**: il parametro che contiene le informazioni sulla rotta corrente, inclusi eventuali parametri passati durante la navigazione. Ogni volta che si naviga verso una nuova schermata, ruote viene aggiornato con i dettagli di quella specifica rotta
- **navigation**: parametro necessario per la navigazione tra le varie pagine dell’applicazione e utilizzato nella fase di aggiunta o modifica della spesa per eseguire rispettivamente il ricaricamento della pagina Nuova spesa e il reindirizzamento dell’utente alla pagina Uscite.

Stati

Nuova spesa

- **inserimento**: stato necessario a raccogliere le informazioni sulla spesa; il tipo è *ins*, definito proprio per la raccolta dei dati da inserire ed il cui codice è il seguente:

```
export type ins = {
  importo: string;
  v_simbolo: string;
  v_nome: string;
  v_sigla: string;
  nome_cat: string;
  descrizione: string;
  data: Date;
  tag: string[];
  idSpesa: string;
};
```

Se l’operazione da eseguire è l’inserimento di una nuova spesa, il valore iniziale dello stato sarà il seguente:

```
const [inserimento, setInserimento] = useState<ins>({
  importo: "",
  v_simbolo: "",
  v_nome: "",
  v_sigla: "",
  nome_cat: "",
  descrizione: "",
  data: new Date(),
  tag: [],
  idSpesa: ""
});
```

```
});
```

Se, invece, l'operazione da eseguire è la modifica di una spesa già esistente allora *inserimento* avrà la seguente configurazione, dipendente dai dati ricavati dal database su quella spesa:

```
setInserimento({
    importo: spesa.importo,
    v_simbolo: valuta.simbolo,
    v_nome: valuta.nome,
    v_sigla: valuta.sigla,
    nome_cat: spesa.categoria,
    descrizione: spesa.descrizione,
    data: (new Date(spesa.data)),
    tag: elencoTag,
    idSpesa: spesaId });
```

- **isModifying:** è lo stato settato sulla base dei parametri passati attraverso route; infatti, se quest'ultimi non sono undefined, vuol dire che l'operazione da eseguire è la modifica di una spesa già esistente e quindi *isModifying* verrà settato a true. Come valore di default impostiamo invece false.

Importo

- **selectedValuePicker:** stato contenente il valore selezionato dall'utente dal menu a tendina delle valute.
- **listaValute:** stato contenente le valute prelevate dal database che verranno utilizzate per creare i *Picker Item* del *Picker* con cui l'utente potrà selezionare la valuta desiderata.
- **loadingImporto:** è lo stato relativo al caricamento del componente *Importo* di Nuova spesa; ha come valore iniziale false e viene settato a true solo dopo che sono state prelevate tutte le valute dal database.

Categorie

- **textAggiuntaCategoria:** stato contenente il nome della categoria da inserire.
- **imgAggiuntaCategoria:** stato contenente il path dell'icona selezionata dall'utente in fase di aggiunta della categoria.
- **selectedIcon:** stato contenente il valore di mappatura dell'icona selezionata dall'utente in fase di aggiunta della categoria.
- **isLoadingCategorie:** è lo stato relativo al caricamento del componente *Categorie* di Nuova spesa; ha come valore iniziale false e viene settato a true solo dopo che sono state eliminate dal database tutte le categorie che non sono più associata a nessuna spesa e dopo che sono state prelevate tutte le icone e tutte le categorie dal database.
- **listaCategorie:** stato contenente la lista di tutte le categorie del conto dell'utente prelevate dal database.
- **icone:** stato contenente tutte le icone prelevate dal database che verranno poi passate alla *FlatList* del pop up di aggiunta categoria.
- **modalVisible:** stato per la gestione della visualizzazione del pop up di aggiunta categoria.
- **selectedCategory:** stato contenente la categoria selezionata dall'utente per la nuova spesa.

Tag

- **tagText:** stato contenente il nome del tag da inserire.
- **tagModalVisible:** stato per la gestione della visualizzazione del pop up di aggiunta tag.

- **isLoadingTag:** è lo stato relativo al caricamento del componente *Tag* di Nuova spesa; ha come valore iniziale false e viene settato a true solo dopo che sono stati prelevati tutti i tag dal database.
- **selectedTag:** contiene il tag o i tag selezionati dall'utente per la spesa da inserire.
- **tag:** stato contenente la lista dei tag prelevati dal database.

Data

- **viewDatePicker:** stato per la gestione della visualizzazione del DateTimePicker.
- **data:** stato contenente la data selezionata dall'utente.

Descrizione

- **text:** stato contenente il testo della descrizione inserita dall'utente.

Funzionalità principali

Inserimento di una spesa

La funzionalità principale a cui è designata la pagina è sicuramente l'aggiunta di una nuova spesa. Per far sì che l'inserimento vada a buon fine, l'utente deve inserire obbligatoriamente l'importo, la categoria e la descrizione della spesa, mentre la valuta e la data hanno dei valori di default per cui se l'utente non selezionerà uno dei due campi, o entrambi, la spesa verrà comunque aggiunta, assegnando come data quella odierna e come valuta l'euro. Per quanto concerne i tag, questi sono facoltativi.

Per migliorare l'esperienza utente, abbiamo aggiunto una nuova funzionalità, non prevista dalle specifiche, che consente di selezionare una valuta dell'importo inserito diversa dalla valuta assegnata al conto. Il sistema convertirà automaticamente l'importo nella valuta predefinita scelta dall'utente durante la registrazione. Così facendo, l'utente potrà sapere sempre l'importo speso nella sua valuta, senza la necessità di dover convertire l'importo con altre metodologie prima di inserire la spesa. Ecco il codice della funzione che ci permette di offrire tale funzionalità nella nostra applicazione:

```
export const conversioneValuta = async (valutPartenza, valutaArrivo, cifra) => {
  try {
    const apiKey = '3aacfff7b17f8023a5f12628';
    const risposta = await fetch(`https://v6.exchangerate-
api.com/v6/${apiKey}/pair/${valutPartenza}/${valutaArrivo}/${cifra}`);
    const json = await risposta.json();
    if (json.result === 'success') {
      const p = parseFloat(json.conversion_result.toFixed(2));
      console.log("Valore convertito: " + p);
      return (p);
    } else {
      alert('Error nella conversione della valuta');
    }
  } catch (error) {
    alert('Error: ' + error.message);
  }
};
```

Da specifiche, l'utente deve avere la possibilità di associare alla spesa una o più categorie; la nostra scelta implementativa è stata permettere all'utente di creare delle macro-categorie, quelle che abbiamo denominato appunto categorie, che è obbligatorio associare alla spesa, e delle micro-categorie facoltative, che sono i tag. Quando ad una categoria non saranno più associate spese, questa

verrà rimossa dal database dell'utente, controllo che effettuiamo ogni volta che l'utente entra nella pagina di Nuova Spesa.

Se l'utente desidera inserire una nuova categoria non presente tra quelle selezionabili, può farlo attraverso il pop up dedicato; una volta aggiunta la categoria, per far andare a buon fine l'assegnazione di quest'ultima alla spesa, essa deve essere selezionata, anche se appare già evidenziata.

Per quanto riguarda i tag, anche qui l'utente può decidere di inserirne uno nuovo, attraverso l'apposito pop up, ma questi non saranno eliminati se non più associati ad una spesa.

Quando tutti i campi obbligatori saranno compilati e l'utente cliccherà sul bottone di "Aggiunta spesa", verrà mostrato un Alert che avverte l'utente che l'operazione è stata completata con successo e la pagina verrà ricaricata in modo tale che l'utente ha la possibilità di inserire un'altra spesa e di muoversi in altre sezioni dell'applicazione attraverso il menu.

Modifica di una spesa

Un'altra funzionalità implementata nella pagina di Nuova Spesa è la modifica di una spesa già esistente; infatti, quando l'utente cliccherà sul bottone di "Modifica" del pop up dedicato alla spesa nella pagina delle Uscite, verrà reindirizzato alla pagina di Nuova Spesa, in cui però i componenti saranno già settati con i valori della spesa da modificare. In questo caso il bottone presente al termine della pagina è quello di "Modifica spesa".

Se l'operazione va a buon fine, l'utente vede un Alert di conferma e viene reindirizzato alla pagina "Uscite", dove potrà vedere la spesa con le modifiche apportate.

Mappatura delle immagini

Come già detto in precedenza, una delle funzionalità fornite all'utente è quella di aggiunta di una nuova categoria; per fare ciò, l'utente deve inserire il nome della nuova categoria e selezionare un'icona da associarle. Quando poi clicca sul bottone "Aggiungi categoria", questa oltre ad essere aggiunta tra le categorie selezionabili, deve essere anche inserita nel database. Avendo usato la funzione *getImageFromPath()* per mostrare le immagini ed essendo che questa funzione restituisce un valore di mappatura dell'immagine, abbiamo avuto la necessità di scrivere un'ulteriore funzione per ri-convertire il valore di mappatura nel path dell'immagine, in modo tale da poter inserire quest'ultimo nel database. La funzione dedicata a questa operazione è *getImagePathFromId()* ed è stata fatta sulla base di una mappatura manuale delle immagini; il codice della funzione è il seguente:

```
const getImagePathFromId = (id) => {
  for (const key in imageMapping) {
    if (imageMapping[key].id === id) {
      return imageMapping[key].path;
    }
  }
  return null;
};
```

Un esempio di mappatura delle immagini è invece questo:

```
MinionsJewels: {
  id: MinionsJewels,
  path: '../assets/img/icone_minions/Minions-Jewels.png',
},
```

Tutta la mappatura, che corrisponde alla creazione di un dizionario i cui elementi sono composti dall'id dell'immagine dal corrispettivo path, viene associata alla costante *imageMapping*.

Grafici

Questo componente è responsabile della visualizzazione di grafici che mostrano l'andamento delle spese nel tempo e il confronto tra le spese per categoria e la media delle spese per categoria. Il componente utilizza librerie come *react-native-chart-kit* e *react-native-pure-chart* per disegnare grafici a linee e a barre. Il componente Grafici accetta due props:

- La proprietà `database` è un oggetto che rappresenta il database SQLite;
- La proprietà `idConto` è una stringa che rappresenta l'ID del conto per il quale devono essere visualizzati i grafici e i diagrammi.

Inoltre, il componente permette di rappresentare due tipi di grafici:

- **Andamento Spese Annuo:** Visualizza un grafico a linea che rappresenta l'andamento delle spese totali per ogni mese nell'ultimo anno. Ogni punto sul grafico rappresenta il totale delle spese per un determinato mese. Gli utenti possono interagire con il grafico facendo clic su un punto per visualizzare i dettagli delle spese per quel mese. Tale grafico è stato implementato usando il componente **LineChart** della libreria *chart-kit*;

```
const chartConfigAndamento = {
  backgroundGradientFrom: "#05AF00",
  backgroundGradientTo: "#0055FF",
  color: (opacity = 1) => `rgba(255, 255, 255, ${opacity})`,
  strokeWidth: 3,
  propsForLabels: {
    fontSize: screenWidth * 0.032,
  },
};

<LineChart
  data={dataGraficoAndamento}
  width={windowWidth}
  height={screenHeight * 0.362}
  chartConfig={chartConfigAndamento}
  bezier
  yAxisSuffix={valutaConto}
  style={styles.chart}
  onDataPointClick={handleDataPointClick}
/>
```

- **Confronto Spese Categoria e Media:** Il componente visualizza un confronto tra le spese effettive per categoria e le spese medie per categoria. Utilizzando un grafico a barre fornito da *react-native-pure-chart*, vengono mostrate le spese effettive e medie per ogni categoria. Ogni categoria è rappresentata da una barra colorata e viene mostrata la percentuale di spese rispetto al totale e alla media. Una legenda fornisce un'indicazione chiara dei colori associati a ciascuna categoria. Per soddisfare il requisito sul grafico, ovvero che quest'ultimo deve rappresentare sia la media che la somma delle spese per categoria, è stato utilizzato il componente fornito dalla libreria *chart-kit* **PureChart type='bar'**. Non è stato possibile utilizzare il componente `BarChart` della libreria enunciata nel punto precedente poiché quest'ultimo componente non supporta la rappresentazione di due dati contemporaneamente (ovvero o rappresentiamo la media per ogni categoria o la somma delle spese per ogni

categoria). Utilizzando il componente di PureChart vengono generati due warning, di cui uno è un errore, che però non influenzano il corretto funzionamento dell'applicazione: tali warning sono associati alla limitazione dei componenti della libreria PureChart.

```
<PureChart data={dataGraficoMedia} type='bar' height={screenHeight * 0.25} />
```

Stati del componente

Nelle applicazioni React, lo stato del componente rappresenta i dati privati e dinamici associati a quel componente specifico. Lo stato viene utilizzato per memorizzare informazioni che possono influenzare il rendering del componente. In altre parole, lo stato determina l'aspetto e il comportamento del componente in base ai dati che contiene. Il componente preso in analisi ha i seguenti stati:

- **spesePerAnno:** Questa variabile di stato memorizza un array di oggetti che rappresentano le spese annuali. Ogni oggetto ha le proprietà:
 - ❖ **mese:** Il nome del mese (es. "Gennaio");
 - ❖ **totale:** La spesa totale per il mese.
- **spesePerCategoria:** Questa variabile di stato memorizza un array di oggetti che rappresentano le spese per categoria. Ogni oggetto ha le proprietà:
 - ❖ **categoria:** Il nome della categoria (es. "Cibo");
 - ❖ **spesaCategoria:** La spesa totale per la categoria;
- **spesePerCategoriaMedia:** Questa variabile di stato memorizza un array di oggetti che rappresentano le spese medie per categoria. Ogni oggetto ha le proprietà:
 - ❖ **categoria:** Il nome della categoria (es. "Cibo");
 - ❖ **spesaCategoria:** La spesa media per la categoria.
- **isLoading:** Questa variabile di stato è un flag booleano che indica se i dati sono ancora in fase di caricamento dal database.
- **valutaConto:** Questa variabile di stato memorizza il simbolo di valuta per il conto utente.

Queste variabili vengono settate tramite le funzioni descritte nella sezione sottostante.

Funzioni principali

Il componente si occupa di recuperare i dati necessari per generare i grafici. Utilizza le funzioni *caricaSpesePerAnno*, *caricaSpesePerCategoria*, *caricaSpesePerCategoriaMedia* e *ottieniValuta* per ottenere rispettivamente i dati relativi alle spese per l'anno corrente, le spese per categoria, le spese medie per categoria e la valuta del conto. Questi dati vengono quindi utilizzati per creare i grafici. La firma delle funzioni è la seguente:

- **caricaSpesePerAnno(database, idConto):** Carica le spese totali per ogni mese nell'ultimo anno dal database associato al conto specificato;
- **caricaSpesePerCategoria(database, idConto):** Carica le spese totali per ogni categoria dal database associato al conto specificato;
- **caricaSpesePerCategoriaMedia(database, idConto):** Carica le spese medie per ogni categoria dal database associate al conto specificato;
- **ottieniValuta(database, idConto):** Ottiene la valuta associata al conto specificato nel database.

Interazioni con l'utente

Il componente esegue il rendering condizionale del contenuto in base allo stato di caricamento e alla disponibilità dei dati.

- Mentre i dati vengono caricati, viene visualizzato un **indicatore di attività**; è stato necessario l'uso dell'indicator poiché senza questo componente si verificava il seguente problema: le funzioni utilizzate per generare i dati (dati che vengono utilizzati dai componenti per mostrare i grafici) erano più lente rispetto al caricamento del componente stesso e dunque, quando veniva fatto il rendering del componente, veniva generato un errore; utilizzando l'indicator insieme alla variabile di stato questo problema è stato risolto, in quanto i grafici vengono renderizzati solo quando le funzioni hanno effettivamente caricato i dati a loro necessari e dunque la variabile di stato `isLoading` è settata a `false`.

```
if (isLoading) {
  return (
    <View style={styles.containerCaricamento}>
      <ActivityIndicator size="large" color="#0000ff" />
      <Text style={styles.textCaricamento}>Caricamento...</Text>
    </View>
  );
}
```

- Se non sono disponibili dati per un grafico specifico, viene visualizzato un messaggio informativo.

Questo garantisce un'esperienza utente fluida anche durante il processo di caricamento. Inoltre, quando si clicca su un punto del grafico dell'andamento delle spese, viene mostrato un popup con dettagli sul mese selezionato, inclusi il mese stesso e il totale delle spese.

```
const handleDataPointClick = ({ index, value }) => {
  const mese = dataGraficoAndamento.labels[index];
  Alert.alert('Dettagli', `Mese: ${mese}\nImporto Totale:
  ${value}${valutaConto}`, [{ text: 'OK' }]);
};
```

Media

Il componente Media calcola e visualizza la spesa media per un periodo di tempo selezionato (giorno, mese, anno). In particolar modo permette all'utente di vedere la spesa media del giorno corrente, mese corrente e anno corrente. Il componente riceve le stesse prop discusse nel componente precedente (Grafici).

Importazioni

I moduli necessari per il corretto funzionamento del componente includono i componenti di base di React Native, il selettore di valori (Componente **Picker** da `@react-native-picker/picker`: Per consentire agli utenti di selezionare le opzioni) e due funzioni personalizzate, *calcolaMedia* e *ottieniValuta*, importati da uno script esterno.

Stati del componente

Le variabili di stato definite sono:

- **opzione**: Variabile di stato che contiene l'opzione attualmente selezionata (Giorno, Mese, Anno) per il calcolo della media.
- **media**: Variabile di stato che contiene il valore della spesa media calcolata, aggiornato dinamicamente in base all'opzione selezionata e ai dati recuperati.
- **valuta**: Variabile di stato che rappresenta il simbolo della valuta associato al conto per cui viene calcolata la media.
- **isLoading**: Variabile di stato per indicare se i dati sono in fase di caricamento (mostra un indicatore di caricamento).

Funzioni principali

Il componente utilizza le funzioni *calcolaMedia* e *ottieniValuta* per recuperare i dati necessari dal database SQLite. La firma di tali funzioni è la seguente:

- **ottieniValuta(database, idConto)**: Ottiene la valuta associata al conto specificato nel database;
- **calcolaMedia(database, idConto, opzione)**: La funzione *calcolaMedia* calcola la spesa media giornaliera, mensile o annuale in base all'opzione selezionata dall'utente. La funzione utilizza una query SQL per recuperare i dati dal database e restituisce la spesa media.

Interazioni con l'utente

Il componente come abbiamo accennato utilizza il componente **Picker** della libreria `@react-native-picker/picker` per consentire all'utente di selezionare l'opzione per la quale desidera visualizzare la spesa media (giornaliera, mensile o annuale). Quando l'utente seleziona un'opzione, viene chiamata la funzione `onChange`, che aggiorna la variabile di stato `opzione` e avvia il caricamento dei dati.

```
<Picker
  selectedValue={opzione}
  onValueChange={onChange}
  style={styles.picker}
  itemStyle={styles.pickerItem}>
  <Picker.Item label="Giorno" value="Giorno" />
  <Picker.Item label="Mese" value="Mese" />
  <Picker.Item label="Anno" value="Anno" />
</Picker>
```

Il componente utilizza il componente **ActivityIndicator** della libreria *react-native* per visualizzare un indicatore di caricamento durante il recupero dei dati. Quando i dati sono pronti, il componente visualizza la spesa media e la valuta corrispondente.

Grafica

Il componente incorpora elementi visivi per migliorare l'esperienza utente e fornire un'interazione più coinvolgente.

Vengono aggiunte due immagini che grazie al bordo del contenitore del componente generano un effetto ottico “giocosso”.

Intervallo

Questo componente consente di visualizzare l'intervallo di spesa sostenuto per un determinato conto in un periodo selezionato. Come per i precedenti componenti, anche quest'ultimo riceve le stesse prop.

L'utente può specificare le date di inizio e di fine tramite **DatePicker** e visualizzare la spesa minima e massima registrata nel periodo selezionato. È inoltre possibile visualizzare la categoria associata alla spesa minima e massima (solo la prima categoria risultante dall'interrogazione del database).

```
<DateTimePicker
  timeZoneName={'Europe/Rome'}
  value={dataFine}
  maximumDate={new Date()}
  mode="date"
  display="calendar"
  onChange={(event, selectedDate) => onChangeData(false, selectedDate)}
/>
```

All'interno dello stesso file abbiamo altri due componenti:

- **ModalContent**

Il componente utilizza il componente **Modal** della libreria *react-native* per visualizzare una finestra di dialogo contenente le informazioni sulle spese minime e massime. Quando l'utente clicca su una delle spese, viene visualizzata la finestra di dialogo. Il contenuto del modal è definito dalla prop *content*. Se *content* è uguale a 'N/A', il modal non viene visualizzato.

```
const ModalContent = ({ modalVisible, onClose, content }) => {
  if (content === 'N/A') {
    return null;
  }

  return (
    <Modal
      animationType="slide"
      transparent={true}
      visible={modalVisible}
      onRequestClose={onClose}>
      <View style={styles.centeredView}>
        <View style={styles.modalView}>
          <Text style={styles.modalText}>{content}</Text>
          <Pressable
            style={[styles.button, styles.buttonClose]}
            onPress={onClose}>
            <Text style={styles.textStyle}>Nascondi</Text>
          </Pressable>
        </View>
      </View>
    </Modal>
  );
};
```

- CalendarButton

Questo componente è un button personalizzato che consente all'utente di aprire il DatePicker.

Dipendenze

Il componente Intervallo richiede diverse librerie e componenti per funzionare correttamente:

- **DateTimePicker** da @react-native-community/datetimepicker - Per consentire la selezione della data;
- **Expo Vector Icons**: Per le icone utilizzate nel componente, in particolare l'icona del calendario;
- **Immagini**: Diverse immagini di Minion utilizzate per rappresentare categorie di spesa;
- **Funzioni personalizzate**: Per il calcolo delle spese minime e massime (*calcolaSpesaMinMax*) e per ottenere la valuta (*ottieniValuta*).

Funzioni principali

La funzione *calcolaSpesaMinMax* viene chiamata all'avvio del componente per recuperare le spese minime e massime per l'intervallo di date selezionato. La funzione utilizza una query SQL per recuperare i dati dal database e restituisce un oggetto contenente le informazioni sulle spese minime e massime, nonché le categorie corrispondenti.

Stati del componente

Il componente Intervallo utilizza diverse variabili di stato per gestire le informazioni relative alle date, alle spese e alle categorie:

1. **dataInizio**: Memorizza la data di inizio del periodo selezionato.
2. **dataFine**: Memorizza la data di fine del periodo selezionato.
3. **spesaMinima**: Memorizza il valore della spesa minima registrata nel periodo selezionato.
4. **spesaMassima**: Memorizza il valore della spesa massima registrata nel periodo selezionato.
5. **categoriaSpesaMin**: Memorizza la categoria associata alla spesa minima.
6. **categoriaSpesaMax**: Memorizza la categoria associata alla spesa massima.
7. **pathMin**: Memorizza il percorso dell'immagine associata alla categoria della spesa minima.
8. **pathMax**: Memorizza il percorso dell'immagine associata alla categoria della spesa massima.
9. **valuta**: Memorizza la valuta associata al conto per cui vengono calcolate le spese.
10. **modalVisibleMinimo**: Determina se il modal contenente la categoria della spesa minima è visibile.
11. **modalVisibleMassimo**: Determina se il modal contenente la categoria della spesa massima è visibile.
12. **isLoading**: Determina se il componente è in fase di caricamento dei dati.

Immagini delle categorie

React Native presenta alcune limitazioni, in particolare con il componente *Image*. Questo componente non accetta valori dinamici per l'attributo *source*. In altre parole, *Image* richiede che tutte le risorse siano precaricate prima del rendering del componente. Questo significa che non è possibile cambiare dinamicamente la risorsa di un *Image* se questa non è già disponibile nel bundle dell'app.

Per risolvere questo problema, è stato adottato il seguente approccio:

1. Importazione delle Immagini in un File JS

Tutte le immagini necessarie sono importate in un singolo file JavaScript. In questo file, è definita una funzione *getImageFromPath()* che prende il percorso (path) della risorsa necessaria e

restituisce il riferimento all'immagine precaricata. Questo riferimento può poi essere utilizzato per il rendering dell'immagine.

2. Utilizzo nel Componente Intervallo

Nel componente Intervallo, viene chiamata la funzione *getImageFromPath()* con il percorso della risorsa necessaria. Il percorso è ottenuto dal risultato della funzione *calcolaSpesaMinMax()*.

Il valore restituito dalla funzione *getImageFromPath()* viene assegnato a una variabile.

```
const imageToShowMin = pathMin ? getImageFromPath(pathMin) : null;
const imageToShowMax = pathMax ? getImageFromPath(pathMax) : null;
```

Questa variabile viene quindi utilizzata come valore della prop *source* del componente *Image*, permettendo di visualizzare l'immagine della categoria con la spesa minima e massima in un determinato intervallo di tempo.

```
{imageToShowMax ? (
  <Image source={imageToShowMax} style={styles.imageCategory}/>
) : (
  <Text style={styles.textNA}>N/A</Text>
)}
```

Questo approccio assicura che tutte le immagini siano precaricate e disponibili per il componente *Image*, risolvendo il problema delle risorse dinamiche non disponibili.

Interazione con l'utente

Quando l'utente seleziona una data, viene chiamata la funzione *onChangeData()*, che aggiorna la variabile di stato corrispondente e avvia il caricamento dei dati.

Inoltre, quando l'utente clicca su una delle spese, viene visualizzata la finestra di dialogo corrispondente.

Esempio di utilizzo

1. L'utente seleziona una data di inizio e una data di fine utilizzando i selettori di date.
2. Il componente calcola automaticamente le spese minime e massime per il periodo selezionato.
3. L'utente può visualizzare le categorie delle spese minime e massime.
4. Se l'utente desidera ulteriori dettagli, può cliccare sulle categorie per visualizzare un modale con informazioni aggiuntive.

SpesePerCategoria

Il componente `SpesePerCategoria` visualizza le spese divise per categoria. Questo componente come gli altri componenti utilizza il database passato come prop (`database`) e l'identificatore di conto (`idConto`) per recuperare e visualizzare le informazioni sulle spese. È responsabile della presentazione delle spese suddivise per categorie, offrendo agli utenti una panoramica dettagliata e organizzata delle loro spese finanziarie.

Il codice è composto da tre componenti principali:

- **SpesePerCategoria:** Questo è il componente principale che gestisce il recupero e la visualizzazione delle spese per categoria.
- **renderSpeseCategoria:** Questo componente viene utilizzato per rendere singole righe che rappresentano ogni categoria di spesa.
- **MinionComponent:** Questo componente visualizza semplicemente un'immagine Minion nella parte inferiore dell'elenco delle spese (solo in modalità verticale nel caso non ci siano spese).

```
{orientamento === 'portrait' && (  
    <View style={styles.noSpeseImageContainer}>  
        <Image  
source={require('../assets/Image/miniSorpresa.png')}  
style={styles.noSpeseImage} />  
    </View>  
    )  
)}
```

L'orientamento viene rilevato tramite la funzione `rilevaOrientamento`:

```
useEffect(() => {  
    const rilevaOrientamento = () => {  
        const { height, width } = Dimensions.get('window');  
        setOrientamento(width > height ? 'landscape' : 'portrait');  
    };  
  
    const subscription=Dimensions.addEventListener('change',  
rilevaOrientamento);  
    rilevaOrientamento();  
    return () => subscription?.remove();  
}, []);
```

La struttura del componente principale è la seguente:

- Il componente mostra una lista di spese divise per categoria.
- Le spese vengono visualizzate utilizzando un componente **FlatList**.
- Ogni voce della lista rappresenta una categoria di spese con la percentuale di contributo al totale delle spese.
- Se non ci sono spese, viene visualizzato un messaggio "Nessuna spesa disponibile". Altrimenti, viene visualizzata un'intestazione "Divisione Spese Per Categoria:" seguita da una lista di righe, ognuna rappresentante una categoria di spesa.
- Il componente gestisce il caricamento delle spese e l'orientamento dello schermo.

Dipendenze

Non vengono importati componenti che non fanno parte del framework React-native e di React.

Tipi di dati

Il componente utilizza il tipo di dati *SpesaCategoriaType* per rappresentare le spese divise per categoria. Questo tipo di dati ha tre proprietà:

- **categoria**: stringa che rappresenta la categoria della spesa;
- **percentuale**: numero che rappresenta la percentuale della spesa rispetto al totale;
- **path**: stringa che rappresenta il percorso dell'immagine associata alla categoria.

```
type SpesaCategoriaType = {  
  categoria: string,  
  percentuale: number,  
  path: string,  
};
```

Funzioni Principali

Il componente importa due funzioni personalizzate: *caricaSpesePerCategoriaSezione()* che ha come parametri l'id del conto e il database ed utilizza una query SQL per recuperare i dati delle spese dalla database, calcola il totale delle spese per ogni categoria e restituisce un array di oggetti con il nome della categoria, la percentuale e il percorso dell'immagine; *getImageFromPath()*, descritta precedentemente per mostrare l'immagine associata alla categoria.

La funzione *renderSpeseCategoria()* è utilizzata per renderizzare ogni elemento dell'elenco delle spese. La funzione riceve un oggetto item di tipo *SpesaCategoriaType* e restituisce un elemento JSX che rappresenta la riga della spesa; ogni riga è composta dal nome della categoria, l'icona associata e la percentuale di spesa.

```
const renderSpeseCategoria = ({ item }: { item: SpesaCategoriaType }) => {  
  const path = getImageFromPath(item.path);  
  return (  
    <View style={styles.rigaSpesa}>  
      <View style={styles.categoriaSpesa}>  
        <Image style={styles.categoriaImmagine} source={path} />  
      </View>  
      <View style={styles.percentualeCategorie}>  
        <Text style={styles.spesaCategoriaText}>{item.categoria}</Text>  
        <Text style={[styles.spesaCategoriaText,  
styles.spesaCategoriaTextPercentuale]}>{item.percentuale}%</Text>  
      </View>  
    </View>  
  );  
};
```

Stati del componente

Il componente utilizza due stati: **spesaCat** e **isLoading**. Lo stato *spesaCat* è un array di oggetti *SpesaCategoriaType* che rappresenta le spese divise per categoria. Lo stato *isLoading* è un booleano che indica se il componente è in fase di caricamento. Il componente verifica se lo stato *isLoading* è true

e, in tal caso, visualizza un indicatore di caricamento. Se lo stato *isLoading* è false, il componente verifica se l'array *spesaCat* è vuoto e, in tal caso, visualizza un messaggio di nessuna spesa disponibile. Se l'array *spesaCat* non è vuoto, il componente visualizza un elenco delle spese divise per categoria utilizzando la funzione *renderSpeseCategoria*.

Orientamento

Il componente rileva l'orientamento dello schermo (verticale o orizzontale) e adatta di conseguenza il layout. Ad esempio, una delle immagini dei Minions viene visualizzata solo in modalità verticale.

HomeGraficiStatistiche

Il componente HomeGraficiStatistiche gestisce la presentazione di insight statistici e rappresentazioni grafiche. Questo componente utilizza React Navigation per la navigazione a schede e integra vari componenti personalizzati per la visualizzazione di dati statistici e grafici.

Dipendenze

Il componente importa le seguenti librerie:

- `createMaterialTopTabNavigator` e `createBottomTabNavigator` da `@react-navigation/material-top-tabs` e `@react-navigation/bottom-tabs`: componenti forniti da *React Navigation* per implementare schede a schede sovrapposte e schede a schede posizionate nella parte superiore e inferiore dello schermo, rispettivamente.
- `AntDesign` da `@expo/vector-icons`: libreria per utilizzare icone vettoriali fornite da Ant Design.
- `Font` da `expo-font`: libreria per caricare font personalizzati nell'applicazione Expo.

Funzioni principali

La funzione `Font.loadAsync` carica il font in modo asincrono e aggiorna lo stato dell'applicazione tramite `useState` una volta che il caricamento è completo: esegue il caricamento del font personalizzato 'fredoka-one'. Il componente si basa sul rendering condizionale ovvero viene controllato se il font è caricato (`if (!fontLoaded) { ... }`) e se il font non è caricato, non viene restituito nulla.

```
async function loadFonts() {
  await Font.loadAsync({
    'fredoka-one': require('../assets/fonts/Fredoka-VariableFont_wght.ttf'),
  });
}
```

Funzionamento

L'obiettivo del componente è quello di fornire un'interfaccia di navigazione tra i vari componenti:

- Componente **HomeMediaMinMax**:
Questo componente renderizza due schermate nidificate all'interno di uno `ScrollView`:
 1. La schermata esterna utilizza **ScrollView** per consentire lo scorrimento verticale del contenuto.
 2. La schermata interna utilizza **View** come contenitore:
All'interno del contenitore principale, due `View` con lo stile vengono utilizzate per posizionare al centro i componenti `Intervallo` e `Media`. Questi componenti, importati da altri file, vengono renderizzati passando `database` e `idConto` come props.
- Componente **Statistiche**:
Questo componente restituisce un componente **TabTop.Navigator** a schede sovrapposte utilizzando `createMaterialTopTabNavigator`. Le due schede sono:
 - "Intervallo di Spesa": Questa scheda rendeirezza il componente `HomeMediaMinMax` passando `database` e `idConto` come props.
 - "Spese per Categoria": Questa scheda renderizza il componente `SpesePerCategoria` passando `database` e `idConto` come props. Si presume che `SpesePerCategoria` sia importato da un altro file e gestisca la visualizzazione delle spese per categoria.

```
const TabTop = createMaterialTopTabNavigator();
```

```

<TabTop.Screen name="Intervallo di Spesa">
  {() => <HomeMediaMinMax database={database} idConto={idConto} />}
</TabTop.Screen>
<TabTop.Screen name="Spese per Categoria">
  {() => <SpesePerCategoria database={database} idConto={idConto} />}
</TabTop.Screen>

```

Il componente principale *HomeGraficiStatistiche* gestisce il caricamento del font personalizzato e contiene il tab navigator posizionato nella parte inferiore dello schermo. Restituisce un componente **SafeAreaView** con un all'interno un componente **TabBottom.Navigator** che contiene due schermate: *Statistiche* e *Grafici*. Questi ultimi vengono renderizzati solo quando il font è caricato.

La prima scheda del TabBottom è chiamata "Statistiche" e grazie alla prop *headerShown: false* viene nascosto l'header della scheda. L'icona della scheda viene impostata utilizzando la funzione *tabBarIcon()* che riceve *color* e *size* come parametri e restituisce un'icona *AntDesign* di tipo *linechart*. Il componente restituito all'interno della scheda è *Statistiche*, a cui vengono passati *database* e *idConto* come props. La seconda scheda è chiamata "Grafici". Le props e la logica per l'icona della scheda sono simili alla prima scheda, utilizzando l'icona *AntDesign* di tipo "barchart". Il componente reso all'interno della scheda è *Grafici* a cui vengono passati gli stessi parametri del componente principale.

```

const TabBottom = createBottomTabNavigator();
<SafeAreaView style={styles.tabBarInferiori}>
  <TabBottom.Navigator>
    <TabBottom.Screen name="Statistiche" options={{
      headerShown: false, tabBarIcon: ({ color, size }) => (
        <AntDesign name="linechart" size={size} color={color} />
      ),
    }}>
      {() => <Statistiche database={database} idConto={idConto} />}
    </TabBottom.Screen>
    <TabBottom.Screen name="Grafici" options={{
      headerShown: false, tabBarIcon: ({ color, size }) => (
        <AntDesign name="barchart" size={size} color={color} />
      ),
    }}>
      {() => <Grafici database={database} idConto={idConto} />}
    </TabBottom.Screen>
  </TabBottom.Navigator>
</SafeAreaView>

```


Screen Minions Wallat

Benvenuto in Minions Wallet

Username:

Mail:

Password:

NomeConto:

Valuta:
EUR €

Registrati

Se hai già un account clicca qui

Figure 1 Scherma Registrazione

Effettua il LOGIN

Username:

Password:

Accedi

Non hai un account? Registrati qui.

Figure 2 Schermata Login

Dashboard

 **3946.80 €**
Totale spese al 5/6/2024

25 ▼ **Nuova Spesa**

	Colazione al bar 30/05/2024	- 35.8 €
	Costume da bagno 30/05/2024	- 55.8 €
	Rifornimento auto 30/05/2024	- 40.75 €
	Festa di compleanno 30/05/2024	- 30.8 €
	Spesa speciale 25/05/2024	- 40.25 €

Figure 3 Schermata Dashboard

Inserisci Nuova Spesa

Inserire la causale :

Categoria :

Abbigli...

Data :

5/6/2024

Importo :

Valuta :

€

Conferma

Figure 4 Schermata Spesa Rapida

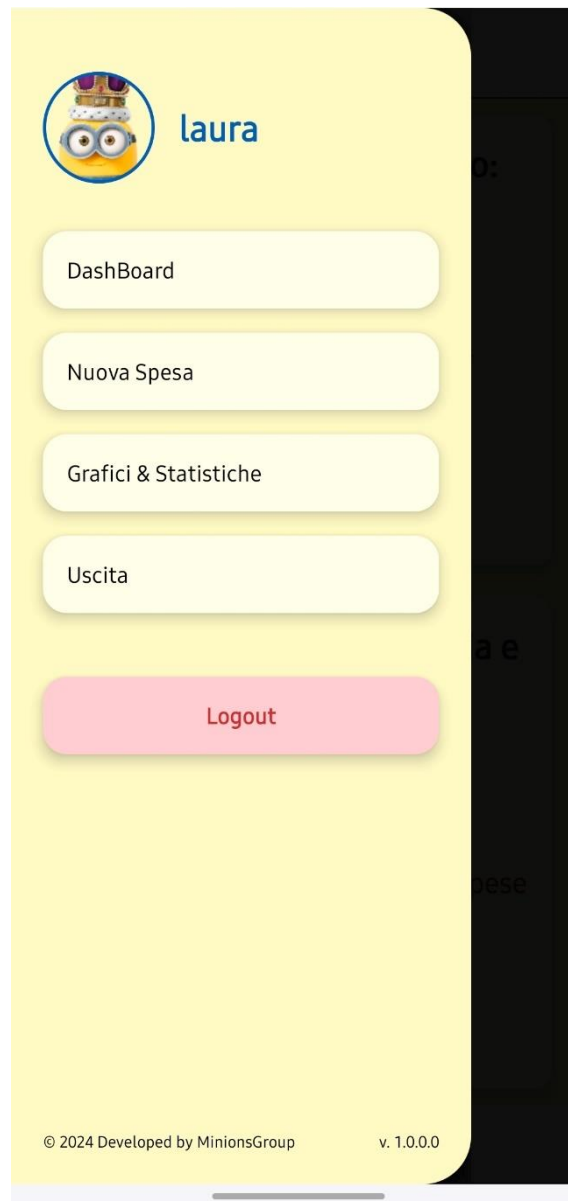


Figure 6 Menu

Aggiungi Spesa

Inserisci l'importo e scegli la valuta

0

€-Euro(...)

Scegli la categoria

Alimentazione

Trasporti

Regali

Viaggi

Svago

Altro

Cibo

Abbigliamento

Intrattenimento

Salute

Casa

+

Inserisci un nuova categoria

Inserisci la descrizione

Scrivi qui la descrizione...

Inserisci la data

Hai effettuato la spesa il 5/6/2024

Seleziona i tag o aggiungine altri

+

Urgente

Regalo fidanzato

Aggiungi Spesa

Figure 5 Schermata Nuova Spesa

42



Figure 7 Schermata Media e Categoria Spesa Max E Min



Figure 8 Schermata Grafici



Figure 9 Schermata Suddivisione Spese Per Categoria

Totale Uscite di carlo

Totale: 159.72₹

Filtra per:

Categoria: Tutte ▼

Data Inizio: Mon Jan 01 2024 Data Fine: Tue Dec 31 2024

Descrizione: Regalo per la Befana a Fra
Categoria: Regali Befana
Importo: 159.72
Data: 4/1/2024

Figure 10 Schermata Uscite

Bug Minions

Attenzione! Anche lui è un po' buggato, quindi ci mette un po' a caricare.....

Piano B: [Minions Video](#)

